Product Requirements Document (PRD)

Project: IT Resource & Operations Platform (IROP) **Version:** 1.0 **Date:** November 2026 **Author:** IROP Development Team **Status:** Approved

Table of Contents

1. Problem Statement
2. Goals & Objectives
3. Technology Stack Selection
4. Target Users
5. User Stories
6. Feature Requirements
7. Scope
8. Success Criteria

1. Problem Statement

IT companies ΓÇö especially small-to-mid-sized firms ΓÇö rely on a fragmented collection of spreadsheets, emails, and disconnected tools to manage three critical operational functions:

- **HR Leave Management:** Employees submit leave requests via email or chat; managers track balances in spreadsheets; there is no centralised audit trail or quota enforcement.
- **IT Asset Management:** Hardware and software licenses are tracked in shared documents that quickly become stale, leading to lost equipment, expired licenses, and no clear ownership chain.
- **Office Inventory Management:** Office supplies and spare parts are managed ad-hoc, resulting in stock-outs, over-ordering, and wasted budget.

This fragmentation causes:

Pain Point	Impact
Manual, error-prone tracking	Data inconsistencies, duplicate records, lost items
No single source of truth	Conflicting information across departments
Lack of role-based access	Sensitive data exposed, unauthorised changes
No real-time visibility	Managers make decisions on stale data
Audit gaps	Compliance risk, inability to reconstruct history

IROP addresses these problems by providing a **unified, role-based web platform** that consolidates leave management, asset tracking, and inventory control into a single application with proper authentication, authorization, and data integrity.

2. Goals & Objectives

Primary Goal

Deliver a production-quality, full-stack web application that enables IT companies to manage employee leave, hardware/software assets, and office inventory from a single dashboard.

Objectives

#	Objective	Measurable Outcome
O-1	Centralise leave management	100% of leave requests flow through the system with automated quota enforcement
O-2	Provide real-time asset visibility	All assets tracked with current status and assignment; searchable within 2 seconds
O-3	Eliminate inventory blind spots	Low-stock alerts surface items at or below threshold automatically
O-4	Enforce role-based access control	Three distinct roles (Employee, Manager, Admin) with server-enforced permissions
O-5	Demonstrate production engineering	Dockerised deployment, CI/CD pipeline, automated testing, clean API design
O-6	Enable rapid onboarding	Seed data and demo logins allow reviewers to explore every feature in < 5 minutes

Engineering & Architectural Objectives

- Implement production-grade full-stack workflows across HR leave, asset lifecycle, and inventory management.
- Model organizational hierarchies and audit trails (reporting managers, direct reports, asset custodians).
- Enforce industry-standard authentication (JWT + salted bcrypt) with server-side RBAC guards.
- Apply modern DevOps best practices: multi-stage Docker containerisation, CI/CD pipelines, automated testing, and zero secrets leakage.

3. Technology Stack Selection: Why MERN is Best for IROP

The platform is architected on the **MERN Stack (MongoDB, Express.js, React, Node.js)** enhanced with **Strict TypeScript**. This stack was chosen over traditional alternatives (such as Python/Django, Java/Spring, or PHP/Laravel) for the following architectural reasons:

Layer	Component	Why It Was Chosen for IROP
Frontend	**React 18 (TypeScript & Vite)**	Operational portals demand fast, reactive user interactions. React's component-driven architecture allows instant UI state updates (filtering 100+ assets, adjusting inventory counts on the fly) without full-page reloads. Coupled with TanStack Query, it provides automatic caching and optimistic UI mutations.
Backend	**Node.js & Express 5**	Node's non-blocking, asynchronous event loop delivers exceptional throughput for concurrent API operations (e.g., simultaneous employee leave requests during peak holiday seasons). Express offers a lightweight, unopinionated routing engine where enterprise security middleware (Helmet, Rate Limiting, CORS) is easily orchestrated.
Database	**MongoDB (Atlas & Mongoose)**	IT assets and operations are naturally semi-structured: hardware items have MAC addresses and serial numbers, while software licenses have seat counts and activation keys. MongoDB's flexible BSON document model accommodates these polymorphic structures seamlessly. Furthermore, MongoDB's atomic `$inc` operators allow safe, race-condition-free quota deductions and stock increments without table-locking overhead.
Language	**End-to-End TypeScript**	By utilizing TypeScript across both frontend and backend, the team eliminates language context-switching and enables shared data contracts (Zod validation schemas, API DTO interfaces), guaranteeing compile-time type safety across network boundaries.

---

4. Target Users

4.1 Employee

Attribute	Detail
Who	Any staff member of the IT company
Needs	Apply for leave, view own quota balances, see assigned assets
Tech comfort	Basic web literacy; expects intuitive forms and clear feedback
Access level	Own data only

4.2 Manager

Attribute	Detail
Who	Team lead or department head responsible for direct reports
Needs	Review and action team leave requests, view asset and inventory data
Tech comfort	Moderate; expects queue-based workflows and at-a-glance dashboards
Access level	Own data + team leave approval + read-only assets/inventory

4.3 Admin (System Administrator)

Attribute	Detail
Who	Operations or IT admin responsible for platform configuration
Needs	Full control over users, assets, inventory; ability to act on any leave request
Tech comfort	High; comfortable with CRUD interfaces and user management
Access level	Full read/write across all modules

5. User Stories

5.1 Employee Stories

ID	Story	Acceptance Criteria
US-E1	As an employee, I want to **apply for leave** by selecting a type, date range, and reason so that my manager can review and approve it.	Form validates dates, computes weekday count, checks remaining balance, and creates a `pending` request.
US-E2	As an employee, I want to **view my leave balance** (Total / Used / Remaining per type) so that I know how many days I have left.	Dashboard displays all four leave-type quotas with correct remaining = total Γêæ used.
US-E3	As an employee, I want to **cancel a pending leave request** so that I can withdraw it before it is reviewed.	Only `pending` requests show a cancel option; cancellation sets status to `cancelled` without affecting quota.
US-E4	As an employee, I want to **see my assigned assets** so that I know which hardware and software is linked to me.	Asset list shows only assets where `assignedTo` matches the logged-in user.
US-E5	As an employee, I want to **view my leave history** with status badges so that I can track past and pending requests at a glance.	History table shows all own leave requests sorted by date with colour-coded status badges.

5.2 Manager Stories

ID	Story	Acceptance Criteria
US-M1	As a manager, I want to **see pending leave requests from my direct reports** so that I can act on them efficiently.	Leave approvals page shows only requests from users whose `managerId` matches the logged-in manager.
US-M2	As a manager, I want to **approve or reject a team member's leave** with an optional note so that the employee is informed of the decision and reason.	Approve deducts quota atomically; reject does not deduct; both update status and record `reviewedBy`, `reviewedAt`, `reviewNote`.
US-M3	As a manager, I want to **view all assets** (read-only, filterable, searchable) so that I can see what hardware and software the company owns.	Asset list displays all assets with status filter and text search on name/serial; no edit/delete controls visible.
US-M4	As a manager, I want to **view inventory levels** (read-only) so that I can be aware of stock situations without needing to request a report.	Inventory table shows all items with quantities, thresholds, and low-stock badges; no +/- controls visible.

5.3 Admin Stories

ID	Story	Acceptance Criteria
US-A1	As an admin, I want to **create user accounts** (with role, department, manager, and quotas) so that new employees can be onboarded without a self-signup flow.	User creation form accepts all required fields; password is hashed; user appears in the user list immediately.
US-A2	As an admin, I want to **manage assets** (create, edit, delete, assign to a user, unassign) so that the company's hardware and software registry stays accurate.	Assign sets status to `in_use`; unassign sets `available`; `in_repair` assets cannot be assigned; all CRUD operations persist correctly.
US-A3	As an admin, I want to **manage inventory** (create, edit, delete, and quick +/- quantity adjustments) so that stock levels reflect reality.	Quantity never goes below 0; items at or below threshold display a low-stock badge; adjustments are optimistic with rollback on error.
US-A4	As an admin, I want to **approve or reject any employee's leave** (including managers' leave) so that no request is blocked by an absent manager.	Admin can act on any `pending` leave regardless of team; same quota rules apply.
US-A5	As an admin, I want to **edit user roles, quotas, and active status** so that I can manage promotions, policy changes, and offboarding.	Role, managerId, department, quotas, and isActive are editable; the system prevents deactivating or demoting the last remaining admin.

---

6. Feature Requirements

Features are prioritised using the **MoSCoW** framework:

- **Must Have (M):** Core functionality; the system is not viable without it.
- **Should Have (S):** Important but not blocking; expected in a complete product.
- **Could Have (C):** Nice-to-have; enhances UX but can be deferred.
- **Won't Have (W):** Explicitly out of scope for v1.0.

6.1 Leave Management System (LMS)

Priority	Feature	Description
M	Apply for leave	Employee selects type (Casual / Sick / Annual / WFH), start date, end date, and reason. System computes weekday count and validates against remaining balance.
M	Approve / Reject leave	Manager approves or rejects direct reports' pending requests with an optional note. Admin can act on any request.
M	Quota tracking	System maintains Total, Used, and Remaining per leave type per user. Quota is deducted only on approval.
M	Leave balance view	Dashboard cards show per-type quota: Total / Used / Remaining.
M	Cancel pending leave	Employee can cancel own leave if status is `pending`. Status changes to `cancelled` without quota impact.
S	Leave history	Filterable list of all own leave requests with status badges (pending: amber, approved: green, rejected: red, cancelled: grey).
S	Overlap detection	Server rejects a new leave request if the date range overlaps an existing pending or approved request for the same user.
S	Weekday-only counting	Days are computed as weekdays (MonΓÇôFri) only; weekends excluded.
C	Team leave calendar	Visual calendar showing team members' approved/pending leave.
W	Half-day leave	Not supported in v1.0.
W	Leave accrual / carry-over	Not supported in v1.0.

6.2 Asset Management System (AMS)

Priority	Feature	Description
M	Asset CRUD	Admin can create, read, update, and delete assets (hardware and software).
M	Assign / Unassign	Admin assigns an asset to a user (sets status `in_use`) or unassigns (sets `available`). `in_repair` assets cannot be assigned.
M	Status filters	Filter asset list by status: In Use, Available, In Repair.
M	Search	Search assets by serial number / license key or item name (text index, debounced on client).
S	My assets view	Employee sees only their own assigned assets.
S	Hardware vs. Software distinction	Asset `kind` field differentiates hardware items from software licenses. Serial number doubles as license key for software.
C	Asset history / audit log	Track assignment and status changes over time.
W	Depreciation tracking	Not supported in v1.0.

6.3 Inventory Management System (IMS)

Priority	Feature	Description
M	Inventory CRUD	Admin can create, read, update, and delete inventory items.
M	Quantity adjustment	Admin uses quick `+1` / `Γêå1` controls; quantity cannot go below 0.
M	Low-stock alerts	Red "Low stock" badge appears when quantity Γëñ threshold.
S	Manager read-only view	Managers can view inventory but cannot modify quantities or items.
S	Category and unit fields	Each item has a category and unit for organisation.
C	Stock history / audit log	Optional `StockLog` model recording each adjustment with timestamp and actor.
W	Purchase order integration	Not supported in v1.0.
W	Supplier management	Not supported in v1.0.

6.4 Cross-Cutting Features

Priority	Feature	Description
M	JWT authentication	Login returns a signed JWT; all protected routes require a valid token. No public signup.
M	Role-based authorisation	Server enforces Employee / Manager / Admin permissions on every endpoint.
M	User management (Admin)	Admin creates, edits, deactivates users; resets passwords.
M	Health endpoint	`GET /health` (unauthenticated) returns server status, DB connectivity, and uptime.
S	Responsive UI	Usable on desktop and tablet viewports.
S	Loading / Empty / Error states	Every data screen handles all three states gracefully.
S	Pagination	Server-side pagination on list endpoints (`page` , `limit`).
C	Dark mode	Toggle between light and dark themes.
W	Email notifications	Not supported in v1.0.

7. Scope

7.1 In Scope

- Full-stack web application (React SPA + Express REST API + MongoDB).
- Three modules: Leave Management, Asset Management, Inventory Management.
- Three roles: Employee, Manager, Admin.
- JWT-based authentication with bcrypt password hashing.
- Seed script for demo data.
- Docker containerisation (client, server, MongoDB) with Docker Compose.
- GitHub Actions CI/CD pipeline (lint, typecheck, build, test, Docker publish to GHCR).
- Comprehensive project documentation (PRD, SRS, API docs, R&D, test report).
- Automated server tests (auth, RBAC, business rules, CRUD).

7.2 Out of Scope

Item	Rationale
Email / SMS notifications	Requires external service integration; deferred to v2.0
File attachments (medical certificates, etc.)	Would require object storage (S3); not needed for MVP
Multi-tenancy	Single-organisation deployment for v1.0
Internationalisation (i18n)	English only for initial v1.0 release
Mobile native app	Responsive web is sufficient
Advanced reporting / analytics	Basic counts on the admin dashboard; no charts or exports
SSO / OAuth integration	JWT with local credentials is sufficient
Audit log for all entities	Optional stock log for inventory; full audit deferred
Half-day or hourly leave	Leave is tracked in full weekdays
Leave accrual, carry-over, or policy engine	Fixed quotas set by admin; no time-based accrual

8. Success Criteria

#	Criterion	Measurement
SC-1	All Must-Have features are implemented and functional	Verified via manual walkthrough and automated tests
SC-2	Server-side business rules pass 100% of automated tests	CI pipeline runs all tests on every push; all pass
SC-3	Role-based access is enforced on every API endpoint	Tests confirm 401 (no token), 403 (wrong role), 200 (correct role) for each protected route
SC-4	Leave quota arithmetic is correct	Approving a 3-day leave deducts exactly 3; rejecting/cancelling deducts 0; balance never goes negative
SC-5	Asset assignment workflow is consistent	Assigning sets `in_use`, unassigning sets `available`, `in_repair` blocks assignment
SC-6	Inventory quantity integrity holds	Quantity ГëÑ 0 at all times; low-stock badge appears at and below threshold
SC-7	Application runs in Docker with a single command	`docker compose up --build` starts all three services; `/health` returns `ok` with `db: connected`
SC-8	CI pipeline passes on clean commits	Lint, typecheck, build, and test steps all succeed
SC-9	Seed script is idempotent	Running `npm run seed` twice yields identical data counts (8 users, 7 leaves, 10 assets, 8 inventory items)
SC-10	Reviewer can explore all features in < 5 minutes	Seed data provides demo logins for each role; README documents the walkthrough

End of Product Requirements Document